

APACHE DORIS 3.0.8

复习题答案

第六章：索引、缓存与高并发查询

20 道综合题参考答案。每题包含一句话结论、底层因果链、得分关键词与常见误区。

配套资料：notes-06

基准版本：Apache Doris 3.0.8

整理日期：2026-08

答案导航与评分

建议先口述再核对。每题 10 分：结论 2 分，结构或机制 4 分，因果链 2 分，边界或验证 2 分。

题号	主题	关键词
1-3	索引总体与有序性	本地跳过、排序、前缀
4-6	ZoneMap 与 Bloom	Min/Max、假阳性、生命周期
7-10	集合与文本索引	Bitmap、Posting、NGram、下推
11	验证工具	EXPLAIN、Profile
12-18	高并发点查	MOW、行存、短路、缓存、Prepared
19-20	系统诊断与建模	瓶颈转移、物理拆分

高质量回答结构



1. 为什么 Doris 的索引体系更强调数据跳过，而不是中央全局 B+Tree？

因为 Doris 是分布式列式 OLAP：先让每个并行扫描任务跳过大量无关数据，通常比维护一个跨节点全局行定位树更符合吞吐和扩展目标。

表被拆成 Partition、Tablet、Segment 和 Page，并分布在多个 BE。FE 负责宏观裁剪，BE 使用与本地数据共同维护的前缀、ZoneMap、Bloom 或倒排结构缩小候选。这样索引跟随数据副本扩展，不需要把所有写入和查询汇聚到中央结构。主键点查是例外，它有更严格的专用路径。

得分关键词 分布式列存；本地索引；Partition；Tablet；Segment；数据跳过

常见误区：把 Doris 索引理解成 MySQL 给每列创建独立全局 B+Tree。两者的主要查询形态、数据布局和扩展目标不同。

2. 排序键与前缀索引分别承担什么职责？

排序键决定行的物理聚集顺序；前缀索引从该有序数据抽取稀疏短键，负责快速导航到可能命中的连续范围。

没有有序性，稀疏导航项无法把查询映射到窄范围。排序键还会让相近值落在相近 Page，改善压缩和 ZoneMap；前缀索引则利用前 36 字节左右的 Key 前缀减少扫描。前者是物理布局，后者是建立在布局上的导航结构。

得分关键词 物理顺序；稀疏索引；Short Key；36 字节；连续范围

常见误区：认为排序键只影响 ORDER BY 输出，或者把前缀索引当成可以独立选择任意列的二级索引。

3. 为什么只按排序键后部列过滤时，前缀索引效果通常较弱？

因为前导列未知时，后部列的相同值可能散落在许多不连续区间，前缀索引无法一次定位窄范围。

例如排序键是 (dt, tenant_id, user_id)。只给 user_id 时，每天、每个租户下面都可能出现该用户，物理位置不连续；给定 dt 和 tenant_id 后，搜索空间变成一个连续前缀，导航能力最强。后部列的次要查询可以考虑倒排等结构。

得分关键词 最左连续前缀；前导列；物理连续；后部列分散

常见误区：机械套用传统单机数据库口诀而不解释物理顺序。关键不是口诀，而是连续区间能否被确定。

4. ZoneMap 为什么在数据聚集时更有效？

因为同类值聚集会让每个 Page 的 Min/Max 区间更窄，查询更容易证明整个 Page 与条件不相交。

随机分布时，一个 Page 可能同时包含很小和很大的值，Min/Max 覆盖范围宽，几乎所有范围条件都只能得到“可能命中”。按相关列排序或保持时间聚集后，不同 Page 的区间边界分明，可以大量跳过。ZoneMap 是粗粒度排除，不负责返回精确 RowID。

得分关键词 Min/Max；数据聚集；窄区间；Page 跳过；粗粒度

常见误区：认为 ZoneMap 记录每个值的位置。它只保存统计边界，区间相交仍需读取和验证。

5. BloomFilter 为什么可以有假阳性，却不能有假阴性？

多个值可能把同一批位设置为 1，导致不存在的值被判为可能存在；但真实插入值对应的位不会凭空变成 0，所以不能被判为一定不存在。

探测时任意一位为 0，就能安全跳过；全部为 1 只进入候选读取，最终由原始数据验证。假阳性增加额外 IO，只影响性能；假阴性会直接丢失正确结果，因此索引实现必须避免。位数组大小、元素数和哈希数共同决定假阳性率。

得分关键词

位数组；多哈希；一定不存在；可能存在；假阳性；正确性

常见误区：认为 BloomFilter 返回精确 RowID，或认为假阳性会让 SQL 返回错误行。残余谓词会做最终验证。

6. 表级 BloomFilter 与 Join Runtime Filter 的生命周期有何不同？

表级 Bloom 索引随数据长期存储并在写入/Compaction 中维护；Runtime Filter 在单次 Join 执行期间动态生成，查询结束即失效。

表级索引描述某列某数据块是否可能包含一个值，服务扫描跳过。Runtime Filter 通常由 Join Build 端根据本次键集合生成，再下推到 Probe 端减少输入。两者可能都用 Bloom 算法，但产生时间、数据来源和服务对象不同。

得分关键词

持久索引；写入维护；查询期；Build/Probe；下推

常见误区：看到 Bloom 字样就认为它们是同一个配置或同一份位数组。

7. Bitmap 索引为什么更适合中低基数字段？

中低基数时每个值对应较大的 RowID 集合，位图压缩和 AND/OR 位运算高效；高基数会产生大量稀疏集合与元数据。

status、region、device_type 的不同值数量有限，可以把多个条件转为位图交并。若 order_id 几乎每行唯一，就接近为每个值维护一个很小的集合，管理成本可能抵消收益。是否使用还要看选择性和真实压缩效果。

得分关键词

RowID 集合；位运算；中低基数；压缩；高基数元数据

常见误区：把低基数等同于一定快。若某状态命中全表大多数行，后续数据读取仍然很重。

8. 倒排索引怎样把全文扫描变成 Posting List 查询？

它把 RowID 到文本的正向关系反转为 Term 到 RowID 列表，查询词项时直接读取并组合对应 Posting List。

写入时文本由 parser 分词，索引记录词项、文档 RowID，短语能力还可能记录位置。MATCH_ANY 做并集，MATCH_ALL 做交集，短语检索检查位置关系；最后只对候选行读取必要列。parser 的语义决定索引里到底有哪些 Term。

得分关键词

Term ; Posting List ; RowID ; 分词器 ; 交集并集 ; 词位置

常见误区：把倒排索引理解成简单保存完整字符串的哈希表，忽略分词与查询语义。

9. 全文倒排索引和 NGram BloomFilter 的文本语义有何不同？

倒排索引回答词项、多个词和短语等语言语义；NGram BF 把字符切片后排除不可能包含子串的数据块，最终仍由 LIKE 验证。

搜索 payment timeout 这两个词适合倒排 Posting List；LIKE '%yme%' 不要求词边界，适合 gram_size=3 的 NGram。NGram 仍有假阳性，gram_size 和 bf_size 影响覆盖、冲突与空间。两者不能只按字段类型选择，必须按查询谓词选择。

得分关键词

全文分词；字符切片；MATCH；LIKE；假阳性；残余验证

常见误区：认为 NGram BF 等价于 Lucene 全文检索，或认为倒排分词天然支持任意中间子串。

10. 为什么索引存在不等于谓词一定能够下推？

索引只能处理实现支持且能在扫描前识别的谓词；函数包裹、隐式类型转换或不兼容语义可能使存储层看不到可探测条件。

例如索引建在原始字符串列上，但查询先做复杂函数再比较，索引结构保存的 Term 或值与表达式结果不是同一语义。优化器必须保证下推不改变结果，否则只能读取后在向量化算子中计算。应通过 EXPLAIN 和 Profile 观察，而不是只看 SHOW CREATE TABLE。

得分关键词

谓词下推；语义等价；函数；CAST；隐式转换；存储层

常见误区：认为建了索引后任何引用该列的表达式都会自动使用索引。

11. EXPLAIN 与 Query Profile 在索引诊断中分别回答什么问题？

EXPLAIN 说明计划打算走哪条路；Profile 说明实际执行中这条路过滤了多少数据、耗时多少以及新瓶颈在哪里。

EXPLAIN 可检查分区裁剪、谓词下推和 SHORT-CIRCUIT。Profile 可查看 RowsRead、RowsReturned、BytesRead、RowsInvertedIndexFiltered 和索引计时。计划命中不等于收益大，必须把索引探测时间、减少的读取量和后续算子一起评估。

得分关键词

计划；实际指标；RowsRead；RowsReturned；索引过滤；计时

常见误区：只凭 SQL 延迟判断索引未生效。缓存、并发、Join 和结果集都可能影响端到端延迟。

12. 为什么列式存储适合分析，却不天然适合宽表整行点查？

分析通常只读少数列并批量计算，列存可以少读、压缩和向量化；宽表点查返回一整行时，要从许多列页随机读取并重新拼装。

同样是读取一行，100 列宽表可能触及很多独立列 Page，IOPS 和解码调度成为瓶颈。行存把多个字段编码在同一行页中，能以更少 IO 返回热列或整行，但会增加冗余。

得分关键词

列裁剪；压缩；向量化；宽表；随机 IO；行拼装

常见误区：简单判断行存总比列存快。大范围聚合若读取行存，反而会读入大量无关字段。

13. 普通 OLAP 链路中的哪些固定成本会限制点查 QPS？

SQL 解析、语义分析、优化器规划、Fragment 构造、协调调度、RPC 和执行器初始化，对只返回一行的请求来说可能比读取数据本身更贵。

分析查询扫描海量数据时这些成本可被摊薄；高 QPS 点查每个请求都重复支付固定成本，FE CPU 往往先饱和。PreparedStatement 复用模板，Short-Circuit 跳过通用计划和调度，单次请求只做主键定位所需工作。

得分关键词

固定成本；FE CPU；解析规划；Fragment；调度；短路

常见误区：只关注磁盘和索引，忽略 FE 解析与协调成本。BE 很闲而 FE CPU 高正是典型信号。

14. 为什么 Short-Circuit 要求完整主键等值条件和 Unique Key MOW？

完整主键等值使系统能够确定分区、分桶、Tablet 和目标键；MOW 使目标键直接对应写入阶段维护的最新版本。

缺少复合 Key 的一部分或使用范围条件，会产生多个候选位置，需要普通扫描。Merge-on-Read 还可能要求读取并合并多个版本，破坏固定短路径。Short-Circuit 本质是利用确定性删除优化、调度和扫描工作，因此适用条件必须严格。

得分关键词

完整 Key；等值；Tablet 定位；最新版本；MOW；确定性

常见误区：把任意 UNIQUE 字段等值过滤都当成主键短路。必须是表 Key 定义和受支持的单表 SQL 形态。

15. Row Store 在底层如何减少宽表点查的随机 IO？

它在列存之外维护一个紧凑行编码列，主键定位后读取一个行存 Page 即可解码多个字段，避免逐列访问和拼装。

数据写入时同时生成列式表示和行式编码。分析查询继续读取列存；点查根据主键索引定位后读取行编码。这个优化改变的是物理读取路径，不是把整张表永久改成纯行存。

得分关键词

行列混存；隐藏行列；单 Page；减少 IOPS；按负载选路

常见误区：认为启用 Row Store 会关闭列存。Doris 保留两种表示，因此才会产生额外空间和写放大。

16. row_store_columns 解决了什么成本问题？

它允许 Doris 3.0 起只为点查接口稳定返回的热列生成行编码，避免把宽表所有冷列都复制到 Row Store。

如果 300 列用户画像接口只返回 8 列，全列行存会放大磁盘、写入和 Compaction；选择性行存把冗余控制在热字段集合。查询请求未包含的行存列仍可走快路径，而不在行存中的返回列可能需要补充列式读取。字段集合应在建表前按 API 契约规划。

得分关键词

3.0；部分列行存；热列；空间；写放大；API 契约

常见误区：把所有可能偶尔查询的列都加入 row_store_columns，重新退化为全列冗余。

17. Page Cache、Row Cache 与 PreparedStatement 分别优化哪一层？

Page Cache 缓存列式数据页，Row Cache 缓存热点主键行，PreparedStatement 缓存会话内的 SQL 解析和表达式准备结果。

前两者位于 BE 读取侧，但服务不同访问单位；PreparedStatement 位于 FE/MySQL 协议侧，减少重复解析和计算。大分析扫描可能给 Page Cache 带来淘汰压力，独立 Row Cache 有助于保护热点点查；三者不能互相替代。

得分关键词

列页；热点行；BE 内存；FE 会话；SQL 模板复用

常见误区：认为开启 Row Cache 就能修复没有走 Short-Circuit 的 SQL，或认为 PreparedStatement 会减少磁盘 IO。

18. 怎样分别证明 Short-Circuit 和服务端 PreparedStatement 已经生效？

用 EXPLAIN 查找 SHORT-CIRCUIT 证明执行快路径；在 fe.audit.log 中查找 Stmt=EXECUTE(...) 证明请求使用服务端 PreparedStatement。

两条证据分别覆盖计划路径和协议执行方式。若没有 SHORT-CIRCUIT，应先查 Unique MOW、Row Store、light_schema_change、完整 Key 等值和 SQL 形态；若没有 EXECUTE，应查 JDBC URL、连接池和对象复用。只看低延迟不能证明能力生效，因为缓存也可能偶然使查询很快。

得分关键词

EXPLAIN；SHORT-CIRCUIT；Audit Log；EXECUTE；JDBC

常见误区：用一次压测平均延迟代替结构证据。性能数字会受缓存、并发和机器状态影响。

19. 为什么索引过滤很多行后，整条 SQL 仍可能很慢？

索引只降低扫描阶段候选集；返回宽列、巨大结果集、Join、聚合、排序、Exchange 或索引自身打开成本仍可能主导总耗时。

瓶颈具有转移性：扫描从 10 秒降到 1 秒后，原来 3 秒的 Shuffle 会变成第一瓶颈。还要比较过滤后的绝对行数，而不是只看过滤百分比。Profile 应沿算子树继续寻找时间、Rows 和 Bytes 的集中点。

得分关键词

候选集；绝对行数；宽列；Join；Exchange；瓶颈转移

常见误区：看到 RowsInvertedIndexFiltered 很大就断言系统异常。它只证明该阶段做了过滤，不代表后续工作消失。

20. 面对同时包含点查、全文检索和聚合的实时业务，怎样进行物理模型拆分？

让同一实时事实流进入不同访问模式表：Unique MOW 行存表服务主键点查，Duplicate 明细表服务分区扫描和文本索引，聚合表或 MV 服务固定看板。

点查表只保留主键与接口热列，使用 Row Store、Short-Circuit 和 PreparedStatement；明细表围绕时间、租户排序并为 remark、URL 等建立倒排或 NGram；固定指标提前聚合。拆分增加数据副本和链路治理，但避免一张表同时支付所有索引与行存成本，也便于资源隔离。

得分关键词

访问模式建模；Unique MOW；Duplicate；倒排/NGram；MV；资源隔离

常见误区：把拆表理解成按字段随意垂直拆分。核心依据是查询路径、更新语义和长期成本，不是表越多越好。

综合自测标准

完成第六章后，应能拿一条慢 SQL 画出从分区到 RowID 的候选缩减过程，并说明每个结构支付了什么成本。对于点查，应能独立验证表属性、SQL 形态、短路标记、PreparedStatement 和缓存层。

能力层级	合格表现	优秀表现
概念	能说出每类索引用途	能解释底层结构和误判边界
设计	会按字段加索引	会从查询模板、选择性和总成本反推
验证	会看 EXPLAIN	会用 Profile 量化过滤、读取和瓶颈转移
点查	会配置 Row Store	能解释 MOW、Short-Circuit、Prepared 和 Cache 分工
治理	能发现无效索引	能按访问模式拆表并评估写入、空间和 Compaction

参考资料

[1] Doris 3.x Index Best Practice

[2] Doris Inverted Index

[3] Doris 3.x Row Store

[4] Doris High-Concurrency Point Query

[5] Release 3.0.8